

Сохранение состояния. Файлы

УРОК №9

План на сегодня



- ☐ Научиться хранить состояния пользователя
- ☐ Поработать с файлами

Сохранение состояния

Состояния ботов

Создать бота-анкету

Запрашиваемые поля:

- имя
- фамилия
- десятилетие рождения

Состояния ботов

Создать бота-анкету

/anketa

Запрашиваемые поля:

- имя
- фамилия
- десятилетие рождения

Ввод имени

Ввод
фамилии

Ввод даты

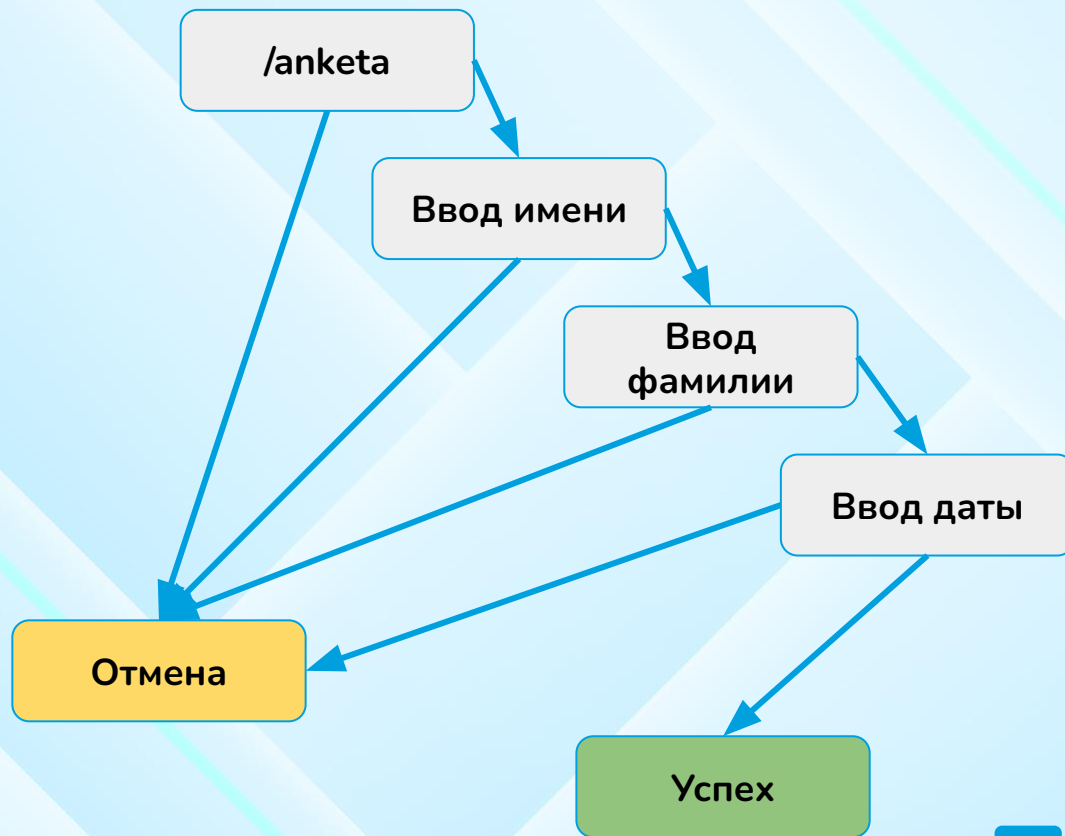
Успех

Состояния ботов

Создать бота-анкету

Запрашиваемые поля:

- имя
- фамилия
- десятилетие рождения



StatesGroup

StatesGroup - определяет набор состояний.

```
class AnketaStates(StatesGroup):  
    name = State()  
    surname = State()  
    birthday = State()
```



StatesGroup

StateFilter(None) - фильтр состояния по умолчанию.

```
@dp.message(StateFilter(None))  
async def empty_message(msg: Message):  
    await msg.answer("Чтобы пройти анкету напишите /start")
```



StatesGroup

state: `FSMContext` - объект управления состоянием пользователя

state.set_state(AnketaStates.name) - выставить состояние пользователя

state.update_data(a="b") - сохранить данные в состояние (эти данные можно будет всегда извлечь из состояния)

state.get_data() - получение словаря данных состояния

state.clear() - очистить состояние от данных и перевести пользователя в состояние по умолчанию



```
@dp.message(Command("start"))
async def start_command(msg: Message, state: FSMContext):
    await state.set_state(AnketaStates.name)
    await msg.answer("Напишите ваше имя")
```

State handler

`AnketaStates.name` - фильтр по сообщений по состоянию.

`state.set_state(AnketaStates.name)` - выставить состояние пользователя

`state.update_data(a="b")` - сохранить данные в состояние (эти данные можно будет всегда извлечь из состояния)

```
@dp.message(AnketaStates.name)
async def name(msg: Message, state: FSMContext):
    await state.update_data(first_name=msg.text)
    await state.set_state(AnketaStates.surname)
    await msg.answer("Напишите свою фамилию")
```



State handler

states

`AnketaStates.birthday` - фильтр по сообщений по состоянию.

`state.update_data(a="b")` - сохранить данные в состояние (эти данные можно будет всегда извлечь из состояния)

`state.get_data()` - получение словаря данных состояния

`state.clear()` - очистить состояние от данных и перевести пользователя в состояние по умолчанию

```
@dp.message(AnketaStates.birthday)
async def name(msg: Message, state: FSMContext):
    await state.update_data(birthday=msg.text)
    data = await state.get_data()
    await msg.answer(
        f"Ваши ответы\nИмя:{data['first_name']}, "
        f"Фамилия: {data['last_name']}, "
        f"День рождения: {data['birthday']}")
    await state.clear()
```



Документы

Документы

`bot.send_photo` - команда для отправки картинки

`bot.send_animation` - отправка gif файлов

`bot.send_document` - отправка файлов

полная запись

```
await bot.send_photo(msg.chat.id, FSInputFile(photo_name))
```

#сокращенная запись

```
await msg.answer_photo(FSInputFile(photo_name))
```





```
@dp.message(Command("photo"))
async def get_photo(msg: Message):
    photo_numer = random.randint(1, 4)
    photo_name = "photo" + str(photo_numer) + ".jpeg"
    await msg.answer_photo(FSInputFile(photo_name))
```


Кеширование документов

`bot.answer_photo` - возвращает объект отправленного сообщения. В нем сохранены картинки и после загрузки у картинок есть `file_id`.
`file_id` - идентификатор файла на сервере telegram.

```
@dp.message(Command("photo"))
async def get_photo(msg: Message):
    photo_numer = random.randint(1, 4)
    photo_name = "photo" + str(photo_numer) + ".jpeg"
    sent_message = await msg.answer_photo(FSInputFile(photo_name))
    print(sent_message.photo)
```



Кеширование документов

`bot.answer_photo` - возвращает объект отправленного сообщения. В нем сохранены картинки и после загрузки у картинок есть `file_id`.

`file_id` - идентификатор файла на сервере telegram.



```
@dp.message(Command("photo"))
async def get_photo(msg: Message):
    photo_numer = random.randint(1, 4)
    photo_name = "photo" + str(photo_numer) + ".jpeg"
    sent_message = await msg.answer_photo(FSInputFile(photo_name))
    print(sent_message.photo)

# [PhotoSize(file_id='AgACAgIAAxkDAAICU2X9vkM37_33strSrf1ylqKk1kNQ...
```

Кеширование документов

`bot.answer_photo` - возвращает объект отправленного сообщения. В нем сохранены картинки и после загрузки у картинок есть `file_id`.

`file_id` - идентификатор файла на сервере telegram.



```
@dp.message(Command("photo"))
async def get_photo(msg: Message):
    photo_numer = random.randint(1, 4)
    photo_name = "photo" + str(photo_numer) + ".jpeg"
    sent_message = await msg.answer_photo(FSInputFile(photo_name))
    print(sent_message.photo[0].file_id)
```

```
# AgACAgIAAxkDAAICVWX9vyFMr54KDXc6iyBoxBbjpC3VAAKD3jEb9P7wS1rBWZ...
```

Кеширование документов

`bot.answer_photo` - может принимать `file_id` в качестве аргумента



```
msg.answer_photo("AgACAgIAAxkDAAICUWX9tVEmMM-Kvybz0Gpk0GFhjdm9AAKC3j  
Eb9P7wSwtA1TbOdMacAQADAgADcwADNAQ", caption="Кешировано")
```

Кеширование документов

Напишем бота, который будет отдавать случайную картинку и кешировать ее.

/photo - загрузить новое фото

/photo_cached - загружать новое фото только если нет file_id в кеше (словаре).



```
photo_cache = {}
```

```
@dp.message(Command("photo"))
async def get_photo(msg: Message):
    photo_number = random.randint(1, 4)
    photo_name = "photo" + str(photo_number) + ".jpeg"
    sent_message = await msg.answer_photo(FSInputFile(photo_name))
```

```
@dp.message(Command("photo_cached"))
async def get_photo(msg: Message):
    global photo_cache

    photo_number = random.randint(1, 4)
    photo_name = "photo" + str(photo_number) + ".jpeg"
    if photo_name in photo_cache:
        await msg.answer_photo(photo_cache[photo_name], caption="Кешировано")
    else:
        message = await msg.answer_photo(FSInputFile(photo_name))
        photo_cache[photo_name] = message.photo[0].file_id
```

План на сегодня



Научиться хранить состояния пользователя



Поработать с файлами

ДОМАШНЕЕ ЗАДАНИЕ

Доработка задач
классной работы.

